

Q. Bayesian Estimation of a User Ability Parameter from Item Responses

An online learning platform presents a user with a sequence of n multiple-choice questions **one at a time**. Each question is either answered correctly or incorrectly, allowing the platform to update its estimate of the user's ability dynamically after every response.

Let Y_i denote the user's response to the i -th item encountered:

$$Y_i = \begin{cases} 1, & \text{if the user answers item } i \text{ correctly,} \\ 0, & \text{if the user answers item } i \text{ incorrectly.} \end{cases}$$

The platform assumes that the probability of a correct response is governed by a two-parameter logistic (2PL) item response model. Specifically, conditional on the user's latent ability parameter θ , the response probability for item i is:

$$P(Y_i = 1 \mid \theta) = p_i(\theta) = \frac{1}{1 + e^{-(a_i(\theta - b_i))}},$$

where $a_i > 0$ is the known discrimination parameter, and b_i is the known difficulty parameter of item i .

Let $\mathbf{y}^{(k)} = (y_1, y_2, \dots, y_k)$ represent the **running vector of observed responses** up to the current step k (where $1 \leq k \leq n$).

Before observing any responses, the platform initializes the user's latent ability estimate with a standard normal prior distribution:

$$f_{\theta}^{(0)}(\theta) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{\theta^2}{2}\right) \quad \text{implying} \quad \theta \sim \mathcal{N}(0, 1).$$

As the user progresses, the posterior distribution at step $k-1$ serves as the prior distribution for step k .

Tasks

- Visualizing the Mechanics:** Plot $P(Y_i = 1 \mid \theta)$ vs θ using Plotly for two distinct values of a_i , where one of those a_i values is paired with three different difficulty values of b_i . Interpret how moving b_i shifts the curve horizontally.
- Sequential Likelihood Contribution:** Write down the likelihood contribution $L(y_k \mid \theta)$ of a *single* new response y_k at step k , given the latent ability θ . Then, write down the joint likelihood function for the running history vector $\mathbf{y}^{(k)}$.
- Mathematical Formulation of the Running Update:** Write down the recursive relationship for the posterior density at step k , denoted $f_{\theta}^{(k)}(\theta \mid \mathbf{y}^{(k)})$, up to a proportionality constant, using the prior state $f_{\theta}^{(k-1)}(\theta \mid \mathbf{y}^{(k-1)})$ and the new observation y_k .
- Dynamic Shifting:** Explain how a correct answer ($y_k = 1$) to a highly difficult item (large b_k) mathematically shifts the peak of the running posterior density distribution relative to the previous step.
- Tracking Certainty and Sharpness:** Explain how the discrimination parameter a_k of the current item alters the variance (or "sharpness") of the distribution during a running update. What happens when a_k is very large versus very small?
- Numerical Implementation of a Running Grid:** Describe a algorithmic approach to numerically approximate and maintain this running posterior density function on a fixed grid of θ -values. Explicitly state how you would perform the sequential normalization step computationally after an item is answered.
- Evaluating Convergence over the Timeline:** Suppose the user's true, hidden latent ability is $\theta_{\text{true}} = 0.75$. Write a Python script that extends your previous grid simulation to track the performance of the running estimators over a sequence of $n = 20$ items.
 - Simulate Responses:** Dynamically generate the user's responses $y_k \in \{0, 1\}$ at each step by comparing a random draw from a Uniform distribution $U(0, 1)$ against the true response probability $p_k(\theta_{\text{true}})$. Give each item a random difficulty $b_k \sim \mathcal{N}(0, 1)$ and a random discrimination $a_k \sim \text{Uniform}(0.5, 2.0)$.
 - Track Estimators:** At each step k , calculate and store the running Posterior Mean ($\widehat{\theta}_{\text{Bayes}}^{(k)}$) and the running Maximum A Posteriori ($\widehat{\theta}_{\text{MAP}}^{(k)}$) estimate.
 - Visualize:** Use Plotly to create a single line chart showing the progression of both estimators from step 0 to 20 . Add a static horizontal reference line at $y = 0.75$ representing θ_{true} .
 - Analysis:** Briefly explain how the distance between your estimators and θ_{true} changes as k increases, and interpret what this implies about the platform's confidence in its measurement.

```
import numpy as np
import plotly.graph_objects as go
```

```

import plotly.io as pio

# Define the 2PL Item Response Function
def p_i(theta, a, b):
    return 1 / (1 + np.exp(-a * (theta - b)))

# Generate range of latent ability values (theta)
theta_vals = np.linspace(-6, 6, 300)

# Configurations:
# Two distinct a_i values (0.5 and 1.5)
# For a_i = 1.5, three distinct b_i values (-2, 0, 2)
curves = [
    {"a": 0.5, "b": 0, "line": dict(dash="dash", width=2.5)},
    {"a": 1.5, "b": -2, "line": dict(dash="solid", width=2.5)},
    {"a": 1.5, "b": 0, "line": dict(dash="solid", width=2.5)},
    {"a": 1.5, "b": 2, "line": dict(dash="solid", width=2.5)},
]

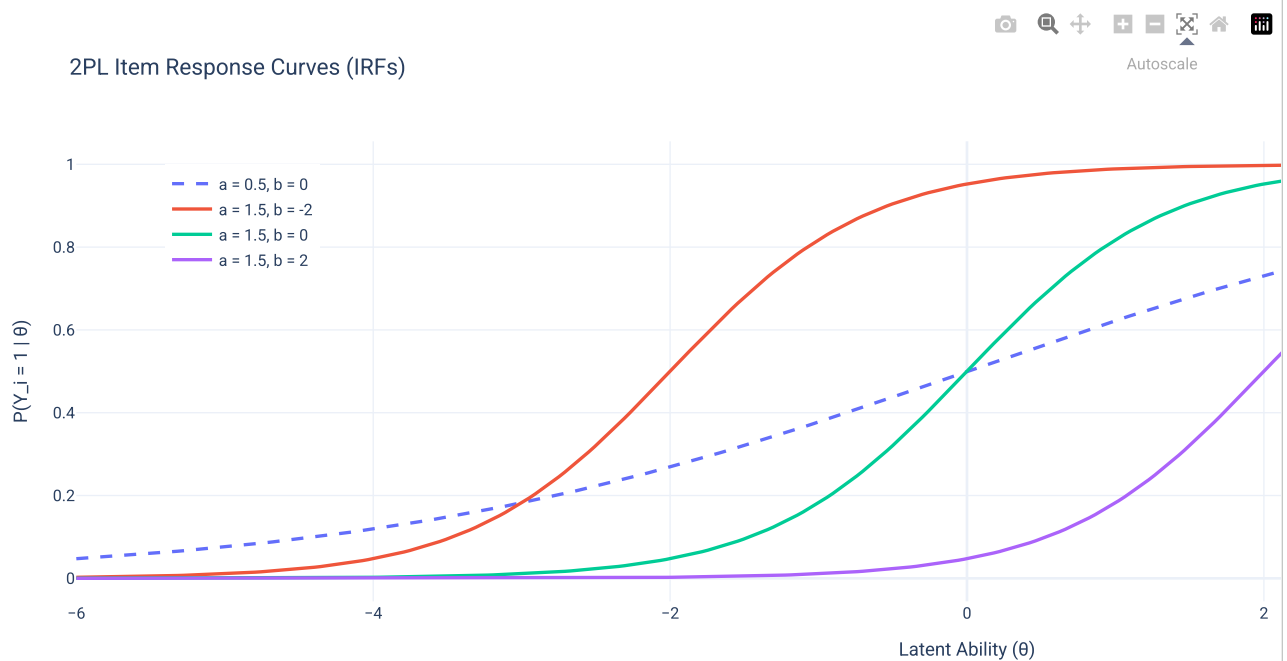
fig = go.Figure()

for curve in curves:
    a, b, line_style = curve["a"], curve["b"], curve["line"]
    p_vals = p_i(theta_vals, a, b)
    fig.add_trace(go.Scatter(
        x=theta_vals,
        y=p_vals,
        mode='lines',
        name=f"a = {a}, b = {b}",
        line=line_style
    ))

fig.update_layout(
    title="2PL Item Response Curves (IRFs)",
    xaxis_title="Latent Ability ( $\theta$ )",
    yaxis_title="P( $Y_i = 1 \mid \theta$ )",
    xaxis=dict(range=[-6, 6]),
    yaxis=dict(range=[0, 1.05]),
    template="plotly_white",
    legend=dict(x=0.05, y=0.95)
)

fig.show()

```



Task 2: Sequential Likelihood Contribution

For a single response $y_k \in \{0, 1\}$ to item k : $L(y_k \mid \theta) = [p_k(\theta)]^{y_k} [1 - p_k(\theta)]^{1 - y_k}$

Assuming conditional independence of responses given θ , the joint likelihood for the running history vector

$\mathbf{y}^{(k)} = (y_1, y_2, \dots, y_k)$ is: $L(\mathbf{y}^{(k)} \mid \theta) = \prod_{i=1}^k [p_i(\theta)]^{y_i} [1 - p_i(\theta)]^{1 - y_i}$

Task 3: Mathematical Formulation of the Running Update

Using Bayes' theorem, the posterior density at step k satisfies the recursive relation: $f_{\theta \mid \mathbf{y}^{(k)}}(\theta \mid \mathbf{y}^{(k)}) \propto L(y_k \mid \theta) f_{\theta \mid \mathbf{y}^{(k-1)}}(\theta \mid \mathbf{y}^{(k-1)})$

Explicitly: $f_{\theta \mid \mathbf{y}^{(k)}}(\theta \mid \mathbf{y}^{(k)}) = \frac{[p_k(\theta)]^{y_k} [1 - p_k(\theta)]^{1 - y_k} f_{\theta \mid \mathbf{y}^{(k-1)}}(\theta \mid \mathbf{y}^{(k-1)})}{\int_{-\infty}^{\infty} [p_k(\theta')]^{y_k} [1 - p_k(\theta')]^{1 - y_k} f_{\theta \mid \mathbf{y}^{(k-1)}}(\theta' \mid \mathbf{y}^{(k-1)}) d\theta'}$

Task 4: Dynamic Shifting

When $y_k = 1$ on a very difficult item (large b_k), $L(y_k = 1 \mid \theta) = p_k(\theta)$ acts as a steep weighting function that is near zero for $\theta \ll b_k$ and ramps up toward 1 for $\theta \gg b_k$.

Multiplying the prior $f_{\theta \mid \mathbf{y}^{(k-1)}}$ by this strongly asymmetrical, monotonically increasing likelihood heavily suppresses the left side of the posterior density. Consequently, the mode (peak) of the posterior density shifts significantly to the right (higher ability estimates).

Task 5: Tracking Certainty and Sharpness

- High discrimination $a_k \approx 0$:** The likelihood function $L(y_k \mid \theta)$ has a sharp step transition around $\theta = b_k$. Multiplying by this steep function sharply truncates one side of the prior, significantly reducing the variance of the posterior and sharpening its peak.
- Low discrimination $a_k \approx 1$:** The likelihood is relatively flat across a wide region of θ , contributing little new information. The posterior's variance changes minimally, and the sharpness remains roughly unchanged.

Task 6: Numerical Implementation of a Running Grid

- Discretization:** Define a fixed grid of M points $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_M)$ spanning a reasonable range (e.g., $[-6, 6]$) with step size $\Delta\theta$.
- Initialization:** Evaluate the prior density on the grid: $q^{(0)}_m = f_{\theta}(\theta_m)$
- Sequential Update Step k :**
 - Compute response probabilities: $p_{k,m} = \frac{1}{1 + e^{-a_k(\theta_m - b_k)}}$
 - Calculate unnormalized posterior: $\tilde{q}^{(k)}_m = [p_{k,m}]^{y_k} [1 - p_{k,m}]^{1 - y_k} \times q^{(k-1)}_m$
 - Normalization:** Sum over the grid to find $S = \sum_{m=1}^M \tilde{q}^{(k)}_m \Delta\theta$, and normalize: $q^{(k)}_m = \frac{\tilde{q}^{(k)}_m}{S}$

```
import numpy as np
import plotly.graph_objects as go

# Set seed for reproducibility
np.random.seed(42)
```

```
# Parameters
```

```

# Parameters
theta_true = 0.75
n_items = 20
grid_points = 500
theta_grid = np.linspace(-6, 6, grid_points)
delta_theta = theta_grid[1] - theta_grid[0]

# Standard normal prior initialization
posterior = (1 / np.sqrt(2 * np.pi)) * np.exp(-0.5 * theta_grid**2)
posterior /= np.sum(posterior * delta_theta)

# Arrays to store estimators
bayes_estimates = [np.sum(theta_grid * posterior * delta_theta)]
map_estimates = [theta_grid[np.argmax(posterior)]]

# Item parameter generation
a_params = np.random.uniform(0.5, 2.0, n_items)
b_params = np.random.normal(0.0, 1.0, n_items)

# Sequential updates
for k in range(n_items):
    a_k = a_params[k]
    b_k = b_params[k]

    # Generate user response based on true ability
    p_true = 1 / (1 + np.exp(-a_k * (theta_true - b_k)))
    y_k = 1 if np.random.uniform(0, 1) < p_true else 0

    # Calculate likelihood on grid
    p_grid = 1 / (1 + np.exp(-a_k * (theta_grid - b_k)))
    likelihood = (p_grid ** y_k) * ((1 - p_grid) ** (1 - y_k))

    # Update and normalize posterior
    posterior = likelihood * posterior
    posterior /= np.sum(posterior * delta_theta)

    # Track estimators
    bayes_est = np.sum(theta_grid * posterior * delta_theta)
    map_est = theta_grid[np.argmax(posterior)]

    bayes_estimates.append(bayes_est)
    map_estimates.append(map_est)

# Plotting with Plotly (after the loop completes)
fig = go.Figure()

steps = list(range(n_items + 1))

fig.add_trace(go.Scatter(
    x=steps, y=bayes_estimates,
    mode='lines+markers', name='Posterior Mean (Bayes)',
    line=dict(color='blue', width=2)
))

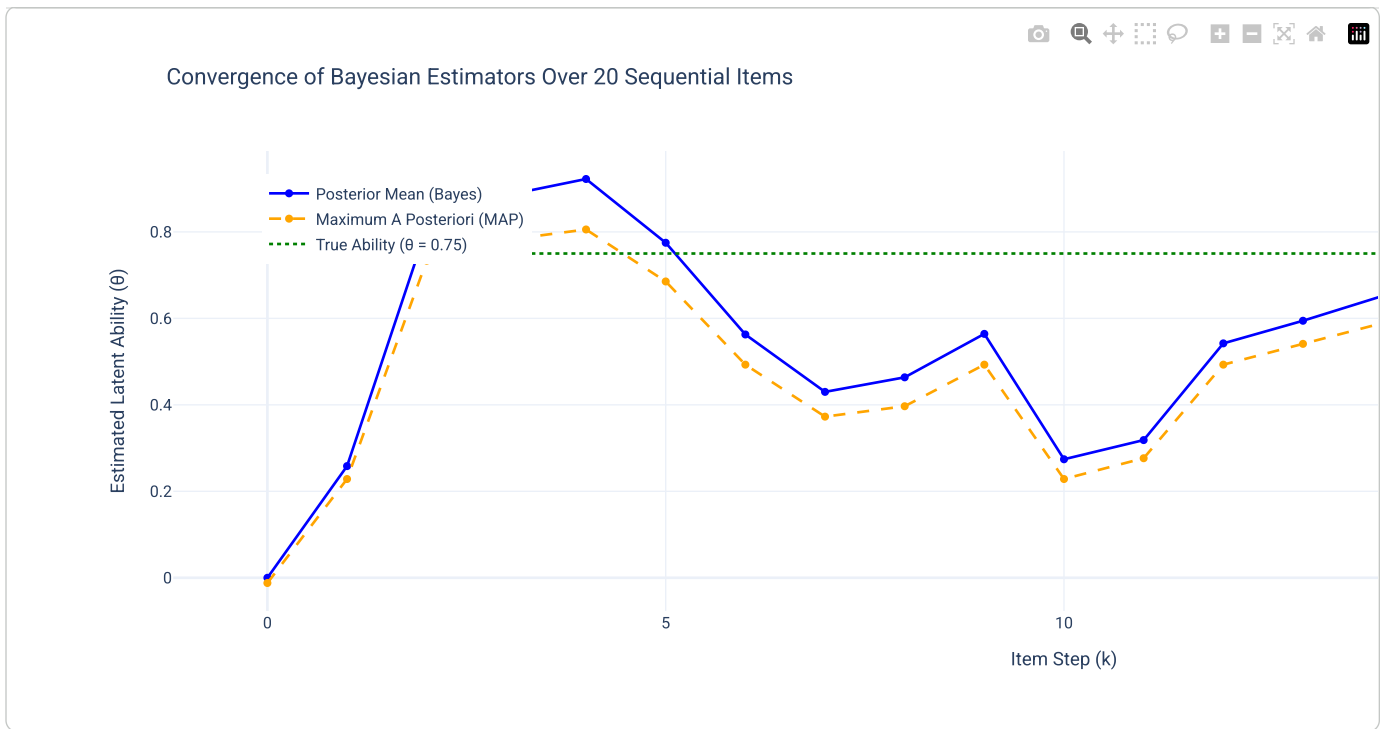
fig.add_trace(go.Scatter(
    x=steps, y=map_estimates,
    mode='lines+markers', name='Maximum A Posteriori (MAP)',
    line=dict(color='orange', width=2, dash='dash')
))

fig.add_trace(go.Scatter(
    x=[0, n_items], y=[theta_true, theta_true],
    mode='lines', name=f'True Ability ( $\theta = \{theta\_true\}$ )',
    line=dict(color='green', width=2, dash='dot')
))

fig.update_layout(
    title="Convergence of Bayesian Estimators Over 20 Sequential Items",
    xaxis_title="Item Step (k)",
    yaxis_title="Estimated Latent Ability ( $\theta$ )",
    template="plotly_white",
    legend=dict(x=0.05, y=0.95)
)

fig.show()

```



Q. Bayesian Tracking of Click-Through Rates (CTR) via Conjugate Beta-Binomial Updates

An e-commerce platform wants to optimize its recommendation engine by dynamically estimating the click-through rate (CTR) of a newly launched advertisement. Since user traffic arrives continuously, the platform updates its belief about the advertisement's performance **one impression at a time** rather than waiting for large batch updates.

Let $\Theta = \theta$ represent the true, hidden conversion rate (probability of a click) of the advertisement, where $\theta \in [0, 1]$.

Let Y_k denote a single user's interaction with the advertisement at time step k :

$Y_k = \begin{cases} 1, & \text{if the user clicks the advertisement} \\ 0, & \text{if the user does not click the advertisement} \end{cases}$

The platform assumes that conditional on the true conversion rate $\Theta = \theta$, each user interaction is an independent Bernoulli trial:

$$P(Y_k = 1 \mid \Theta = \theta) = \theta$$

Let $\mathbf{y}^{(k)} = (y_1, y_2, \dots, y_k)$ represent the **running vector of observed user interactions** up to the current impression step k (where $1 \leq k \leq n$).

Before observing any data, the platform assigns a flexible **Beta distribution** as the initial prior over the unknown parameter Θ :

$$f_{\Theta}^{(0)}(\theta) = \frac{1}{B(\alpha_0, \beta_0)} \theta^{\alpha_0 - 1} (1 - \theta)^{\beta_0 - 1} \quad \text{implying} \quad \Theta \sim \text{Beta}(\alpha_0, \beta_0)$$

where $B(\cdot, \cdot)$ is the Beta function acting as the normalizing constant. Under a sequential framework, the posterior distribution at step $k-1$ serves directly as the prior distribution for step k .

Tasks

1. Structural Probability and Properties Plot the probability density function (PDF) of a $\text{Beta}(\alpha, \beta)$ distribution using Plotly for three distinct parameter pairs:

- Uninformative state: $(\alpha=1, \beta=1)$
- Right-skewed state: $(\alpha=2, \beta=8)$

- Left-skewed state: $(\alpha=8, \beta=2)$

Interpret how changing the balance between α and β shifts the center of mass of the density function over the domain $[0, 1]$.

2. Sequential Likelihood and Joint History

Write down the mathematical likelihood contribution $L(y_k | \theta)$ of a *single* isolated response y_k at step k , given the click probability θ . Following this, express the joint likelihood function for the running history vector $\mathbf{y}^{(k)}$.

3. Closed-Form Analytical Updates (Conjugacy)

Using Bayes' Theorem, derive the recursive algebraic relationship for the posterior density at step k , denoted as $f_{\theta | \mathbf{y}^{(k)}}(\theta | \mathbf{y}^{(k)})$. Prove analytically that the posterior remains in the Beta family (**Beta-Binomial Conjugacy**) by explicitly writing down the closed-form update parameters α_k and β_k as simple arithmetic updates of α_{k-1} and β_{k-1} , and y_k . Also compute the **Posterior Mean** of the latent parameter θ at time step k (i.e. $E[\theta | \mathbf{y}^{(k)}]$).

4. Dynamic Shifting Mechanics

Explain how an observed click ($y_k = 1$) vs. a non-click ($y_k = 0$) shifts the peak of the running density distribution mathematically. Contrast this analytical framework against non-conjugate setups (such as the 2PL IRT model) where numerical grid integration is strictly required.

5. Running Point Estimators

State the exact closed-form equations used to evaluate the following point estimates at step k directly from the updated shape parameters α_k and β_k :

- **Running Posterior Mean** ($\hat{\theta}_{\text{Bayes}}^{(k)}$)
- **Running Maximum A Posteriori** ($\hat{\theta}_{\text{MAP}}^{(k)}$)

6. Performance Tracking and Convergence Analysis

Suppose the advertisement's true, hidden click-through rate is $\theta_{\text{true}} = 0.35$. Write a Python script to track the performance of your closed-form sequential estimators over a timeline of $n = 100$ impressions:

- **Initialize State:** Set the base prior parameters to $\alpha_0 = 1, \beta_0 = 1$ (representing uniform initial uncertainty).
- **Simulate Responses:** Dynamically generate user responses $y_k \in \{0, 1\}$ at each step by comparing a random draw from a Uniform distribution $U(0,1)$ against θ_{true} .
- **Track Estimators:** Loop through each step, update α_k and β_k analytically, and store the computed values for $\hat{\theta}_{\text{Bayes}}^{(k)}$ and $\hat{\theta}_{\text{MAP}}^{(k)}$.
- **Visualize:** Use Plotly to create a single line chart showing the progression of both estimators from step 0 to 100 . Add a static horizontal reference line at $y = 0.35$ representing θ_{true} .
- **Analysis:** Explain how the distance between your estimators and θ_{true} responds as the sampling size k approaches 100 . What does this imply about the accumulation of evidence over time relative to the choice of the initial prior?

```
import numpy as np
import plotly.graph_objects as go
from scipy.stats import beta

# Domain for theta
theta_vals = np.linspace(0.001, 0.999, 500)

# Beta parameter configurations
beta_configs = [
    {"alpha": 1, "beta": 1, "name": "Uninformative: Beta(1, 1)", "color": "purple"},
    {"alpha": 2, "beta": 8, "name": "Right-Skewed: Beta(2, 8)", "color": "red"},
    {"alpha": 8, "beta": 2, "name": "Left-Skewed: Beta(8, 2)", "color": "blue"}
]

fig = go.Figure()

for config in beta_configs:
    a, b_param = config["alpha"], config["beta"]
    pdf_vals = beta.pdf(theta_vals, a, b_param)
```

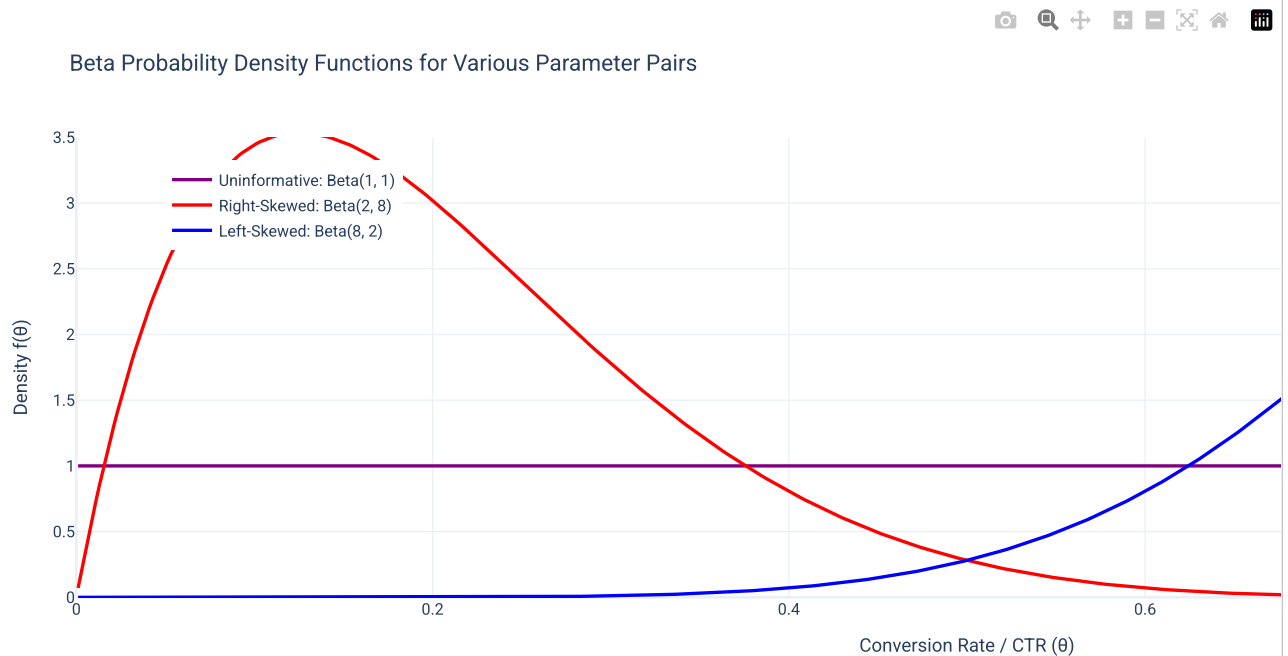
```

fig.add_trace(go.Scatter(
    x=theta_vals,
    y=pdf_vals,
    mode='lines',
    name=config["name"],
    line=dict(color=config["color"], width=2.5)
))

fig.update_layout(
    title="Beta Probability Density Functions for Various Parameter Pairs",
    xaxis_title="Conversion Rate / CTR ( $\theta$ )",
    yaxis_title="Density  $f(\theta)$ ",
    xaxis=dict(range=[0, 1]),
    yaxis=dict(range=[0, 3.5]),
    template="plotly_white",
    legend=dict(x=0.05, y=0.95)
)

fig.show()

```



Task 2: Sequential Likelihood and Joint History

1. Likelihood Contribution of a Single Response

For an isolated observation $y_k \in \{0, 1\}$ at impression step k , the likelihood contribution given the latent conversion parameter $\Theta = \theta$ is modeled as a Bernoulli trial:

$$L(y_k \mid \theta) = \theta^{y_k} (1 - \theta)^{1 - y_k}$$

2. Joint Likelihood of the Running History Vector

Assuming user interactions are conditionally independent given $\Theta = \theta$, the joint likelihood of observing the history vector $\mathbf{y}^{(k)} = (y_1, y_2, \dots, y_k)$ is the product of individual likelihood contributions:

$$L(\mathbf{y}^{(k)} \mid \theta) = \prod_{i=1}^k L(y_i \mid \theta) = \prod_{i=1}^k \theta^{y_i} (1 - \theta)^{1 - y_i}$$

By grouping exponents, this simplifies to:

$$L(\mathbf{y}^{(k)} \mid \theta) = \theta^{\sum_{i=1}^k y_i} (1 - \theta)^{k - \sum_{i=1}^k y_i}$$

Task 3: Closed-Form Analytical Updates & Conjugacy Proof

1. Derivation via Bayes' Theorem

At step k , the prior probability density function is the posterior density from step $k-1$:

$$f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) = \frac{1}{\mathcal{Z}_{k-1}} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$
$$f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) = \frac{1}{\mathcal{Z}_{k-1}} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$

Applying Bayes' Theorem for the new observation y_k :

$$f_{\Theta}(\theta \mid \mathbf{y}^{(k)}) = \frac{L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)})}{\int_0^1 L(y_k \mid \theta') f_{\Theta}(\theta' \mid \mathbf{y}^{(k-1)}) d\theta'}$$
$$f_{\Theta}(\theta \mid \mathbf{y}^{(k)}) = \frac{L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)})}{\int_0^1 L(y_k \mid \theta') f_{\Theta}(\theta' \mid \mathbf{y}^{(k-1)}) d\theta'}$$

Focusing on the unnormalized numerator:

$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{y_k} (1-\theta)^{1-y_k} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$
$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{y_k} (1-\theta)^{1-y_k} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$

Substituting the functional forms:

$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{y_k} (1-\theta)^{1-y_k} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$
$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{y_k} (1-\theta)^{1-y_k} \theta^{\alpha_{k-1}-1} (1-\theta)^{\beta_{k-1}-1}$$

Dropping constant terms independent of θ :

$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{\alpha_{k-1}+y_k-1} (1-\theta)^{\beta_{k-1}+1-y_k-1}$$
$$L(y_k \mid \theta) f_{\Theta}(\theta \mid \mathbf{y}^{(k-1)}) \propto \theta^{\alpha_{k-1}+y_k-1} (1-\theta)^{\beta_{k-1}+1-y_k-1}$$

2. Proof of Beta-Binomial Conjugacy

The functional kernel $\theta^{\alpha-1} (1-\theta)^{\beta-1}$ uniquely identifies a $\text{Beta}(\alpha, \beta)$ distribution. Thus, the posterior distribution stays strictly within the Beta family with closed-form parameter updates:

$$\alpha_k = \alpha_{k-1} + y_k$$

$$\beta_k = \beta_{k-1} + (1 - y_k)$$

Including the normalization constant $\mathcal{Z}_k$, the exact posterior density function is:

$$f_{\Theta}(\theta \mid \mathbf{y}^{(k)}) = \frac{1}{\mathcal{Z}_k} \theta^{\alpha_k-1} (1-\theta)^{\beta_k-1}$$
$$f_{\Theta}(\theta \mid \mathbf{y}^{(k)}) = \frac{1}{\mathcal{Z}_k} \theta^{\alpha_k-1} (1-\theta)^{\beta_k-1}$$

3. Closed-Form Posterior Mean

The expected value of θ at step k is:

$$\mathbb{E}[\theta \mid \mathbf{y}^{(k)}] = \frac{\alpha_k}{\alpha_k + \beta_k} = \frac{\alpha_0 + \sum_{i=1}^k y_i}{\alpha_0 + \beta_0 + k}$$
$$\mathbb{E}[\theta \mid \mathbf{y}^{(k)}] = \frac{\alpha_k}{\alpha_k + \beta_k} = \frac{\alpha_0 + \sum_{i=1}^k y_i}{\alpha_0 + \beta_0 + k}$$

Task 4: Dynamic Shifting Mechanics

1. Mathematical Effect of Observations

- When a Click Occurs ($y_k = 1$):** $\alpha_k = \alpha_{k-1} + 1$, $\beta_k = \beta_{k-1}$. Multiplying the prior density by $L(1 \mid \theta) = \theta$ increases the exponent of θ . This pulls the probability mass and the distribution peak toward 1 (higher CTR).
- When a Non-Click Occurs ($y_k = 0$):** $\alpha_k = \alpha_{k-1}$, $\beta_k = \beta_{k-1} + 1$. Multiplying the prior density by $L(0 \mid \theta) = 1 - \theta$ increases the exponent of $(1 - \theta)$. This pulls the probability mass and the

distribution peak toward 0 (lower CTR).

2. Conjugate Framework vs. Non-Conjugate Models

- **Conjugate Beta-Binomial:** The posterior distribution remains in the same parametric family as the prior. Updates require only simple arithmetic addition ($\mathcal{O}(1)$ computational time), making exact continuous tracking possible without approximation errors.
- **Non-Conjugate Models (e.g., 2PL IRT):** The product of non-linear logistic functions and a Gaussian prior yields a distribution without a simple standard closed form. Updating requires numerical integration (grid evaluation, quadrature) or stochastic approximations (MCMC, Variational Bayes) at every step.

Task 5: Running Point Estimators

Given updated shape parameters α_k and β_k at step k , point estimates are evaluated directly as:

1. Running Posterior Mean (Bayes Estimator)

Minimizes expected squared error loss (Bayesian Minimum Mean Squared Error):

$$\widehat{\theta}_{\text{Bayes}}^{(k)} = \mathbb{E}[\theta \mid \mathbf{Y}^{(k)}] = \frac{\alpha_k}{\alpha_k + \beta_k}$$

2. Running Maximum A Posteriori (MAP Estimator)

Represents the mode (peak) of the posterior density function (for $\alpha_k > 1, \beta_k > 1$):

$$\widehat{\theta}_{\text{MAP}}^{(k)} = \arg\max_{\theta} f_{\theta}(\mathbf{Y}^{(k)}) f_{\theta}(y^{(k)}) = \frac{\alpha_k - 1}{\alpha_k + \beta_k - 2}$$

```
import numpy as np
import plotly.graph_objects as go

# Set seed for reproducibility
np.random.seed(42)

# Parameters
theta_true = 0.35
n_impressions = 100

# Base prior: Beta(1, 1) - Uniform
alpha_k = 1
beta_k = 1

# Track estimators across time steps (including prior step 0)
bayes_estimates = [alpha_k / (alpha_k + beta_k)]
# MAP for Beta(1,1) is undefined/flat, initialized at 0.5
map_estimates = [0.5]

# Sequential Simulation
for k in range(1, n_impressions + 1):
    # Simulate binary response from true underlying rate
    y_k = 1 if np.random.uniform(0, 1) < theta_true else 0

    # Exact Analytical Conjugate Update
    alpha_k += y_k
    beta_k += (1 - y_k)

    # Compute Point Estimators
    bayes_est = alpha_k / (alpha_k + beta_k)
    map_est = (alpha_k - 1) / (alpha_k + beta_k - 2) if (alpha_k > 1 and beta_k > 1) else 0.5

    bayes_estimates.append(bayes_est)
    map_estimates.append(map_est)

# Plotting with Plotly (after the loop completes)
steps = list(range(n_impressions + 1))
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=steps, y=bayes_estimates,
    mode='lines+markers', name='Posterior Mean (Bayes)',
    line=dict(color='blue', width=2)
))

fig.add_trace(go.Scatter(
```

```

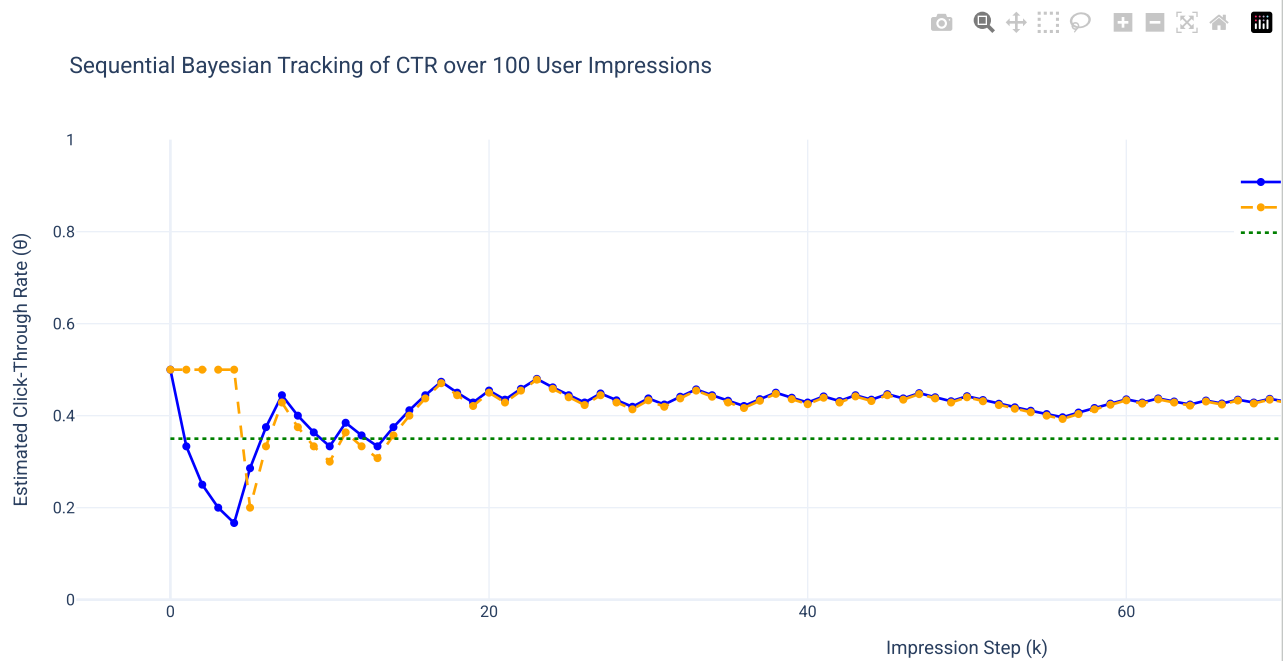
x=steps, y=map_estimates,
mode='lines+markers', name='Maximum A Posteriori (MAP)',
line=dict(color='orange', width=2, dash='dash')
))

fig.add_trace(go.Scatter(
x=[0, n_impressions], y=[theta_true, theta_true],
mode='lines', name=f'True CTR ( $\theta = \{\theta_{\text{true}}\}$ )',
line=dict(color='green', width=2, dash='dot')
))

fig.update_layout(
title="Sequential Bayesian Tracking of CTR over 100 User Impressions",
xaxis_title="Impression Step (k)",
yaxis_title="Estimated Click-Through Rate ( $\theta$ )",
yaxis=dict(range=[0, 1]),
template="plotly_white",
legend=dict(x=0.65, y=0.95)
)

fig.show()

```



Start coding or [generate](#) with AI.